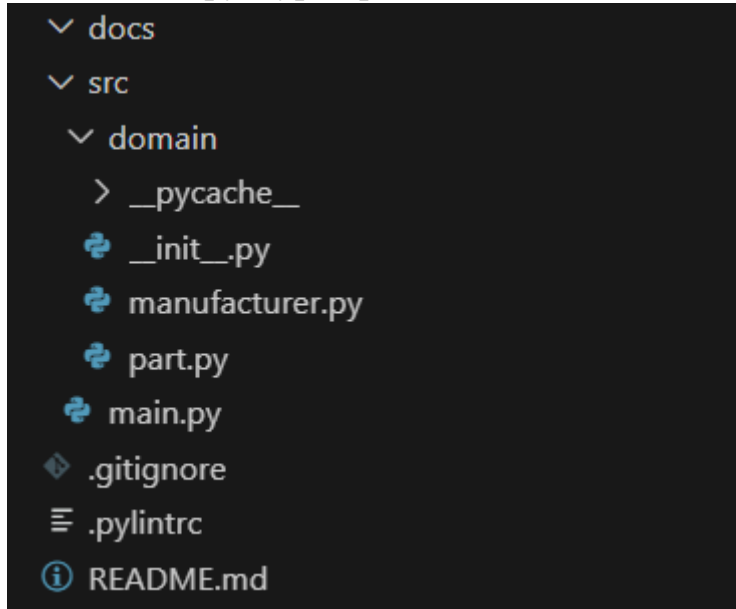


Рубежный контроль №1 по дисциплине ПиКЯП

Никандров Валентин ИУ5-35Б

Вариант №19 («Г»)

Файловая структура проекта:



Листининг кода проекта:

```
"""Main application module"""

from domain import Part, Manufacturer

class PartsManufacturers:
    """Represents a many-to-many relationship between parts and manufacturers.

    Attributes:
        part_id: The unique identifier of the part.
        manufacturer_id: The unique identifier of the manufacturer.
    """

    def __init__(self, part_id: int, manufacturer_id: int):
        self.part_id = part_id
        self.manufacturer_id = manufacturer_id

parts = [
    Part(0, 0, "Поршень двигателя V8", 450, "Двигатель", 2500),
    Part(1, 1, "Турбокомпрессор ТКР-7", 3200, "Система наддува", 18500),
    Part(2, 1, "Катализатор", 2800, "Выхлопная система", 12200),
    Part(3, 2, "Рычаг подвески верхний", 1850, "Подвеска", 4700),
    Part(4, 3, "Фара светодиодная би-ксенон", 950, "Освещение", 8900),
]
```

```

manufacturers = [
    Manufacturer(
        0,
        "Bosch Автокомпоненты",
        "Автомобильные компоненты",
        "Системы впрыска",
        500000,
        "+7 (495) 123-45-67",
        "bosch@example.com",
    ),
    Manufacturer(
        1,
        "Garrett Motion",
        "Автомобильные системы",
        "Турбонаддув",
        200000,
        "+7 (495) 234-56-78",
        "garrett@example.com",
    ),
    Manufacturer(
        2,
        "ZF Friedrichshafen",
        "Автомобильные системы",
        "Подвеска и трансмиссия",
        300000,
        "+7 (495) 456-78-90",
        "zf@example.com",
    ),
    Manufacturer(
        3,
        "HELLA Russia",
        "Автомобильная электроника",
        "Освещение",
        350000,
        "+7 (495) 567-89-01",
        "hella@example.com",
    ),
]

```

```

parts_manufacturers = [
    PartsManufacturers(0, 0),
    PartsManufacturers(1, 1),
    PartsManufacturers(2, 1),
    PartsManufacturers(3, 2),
    PartsManufacturers(4, 3),
    PartsManufacturers(0, 2),
]

```

```

def first_task(original_one_to_many):
    """Print manufacturers starting with 'G' and their parts."""

```

```

print("\nЗадание Г1:")

one_to_many = sorted(original_one_to_many, key=lambda x:
x["manufacturer_id"])
previous_manufacturer_name = ""

for part in one_to_many:
    manufacturer_name = part["manufacturer_name"]
    if not manufacturer_name or manufacturer_name[0] != "G":
        continue
    if manufacturer_name != previous_manufacturer_name:
        previous_manufacturer_name = manufacturer_name
        print(f"\nПроизводитель {manufacturer_name}:")

    part_name = part["name"]
    print(part_name)

def second_task(original_one_to_many):
    """Print the most expensive part for each manufacturer."""
    print("\nЗадание Г2:")

    one_to_many = sorted(
        original_one_to_many, key=lambda x: x["production_cost"], reverse=True
    )
    previous_manufacturers_ids = set()

    for part in one_to_many:
        manufacturer_id = part["manufacturer_id"]
        if manufacturer_id not in previous_manufacturers_ids:
            previous_manufacturers_ids.add(manufacturer_id)

            manufacturer_name = part["manufacturer_name"]
            name = part["name"]
            production_cost = part["production_cost"]

            print(
                f"Производитель: {manufacturer_name}; "
                f"Запчасть: {name}; "
                f"Себестоимость производства: {production_cost}"
            )

def third_task(original_many_to_many):
    """Print manufacturers and their parts sorted by production capacity."""
    print("\nЗадание Г3:")

    many_to_many = sorted(
        original_many_to_many,
        key=lambda x: x["manufacturer_production_capacity"],
        reverse=True,

```

```

)
previous_manufacturer_name = ""

for part_manufacturer in many_to_many:
    manufacturer_name = part_manufacturer["manufacturer_name"]
    if manufacturer_name != previous_manufacturer_name:
        previous_manufacturer_name = part_manufacturer["manufacturer_name"]
        manufacturer_production_capacity = part_manufacturer[
            "manufacturer_production_capacity"
        ]

        print(
            f"\nПроизводитель {manufacturer_name} (Производственная "
            f"мощность {manufacturer_production_capacity}):"
        )

    print(part_manufacturer["part_name"])

def main():
    """Main function."""

    one_to_many = [
        {
            "name": part.name,
            "production_cost": part.production_cost,
            "manufacturer_name": manufacturer.name,
            "manufacturer_id": manufacturer.id,
        }
        for manufacturer in manufacturers
        for part in parts
        if part.manufacturer_id == manufacturer.id
    ]

    first_task(one_to_many)
    second_task(one_to_many)

    many_to_many_temp = [
        {
            "manufacturer_name": manufacturer.name,
            "manufacturer_production_capacity": manufacturer.production_capacity,
            "manufacturer_id": part_manufacturer.manufacturer_id,
            "part_id": part_manufacturer.part_id,
        }
        for manufacturer in manufacturers
        for part_manufacturer in parts_manufacturers
        if manufacturer.id == part_manufacturer.manufacturer_id
    ]

    many_to_many = [
        {

```

```

        "manufacturer_name": part_manufacturer["manufacturer_name"],
        "manufacturer_production_capacity": part_manufacturer[
            "manufacturer_production_capacity"
        ],
        "part_name": part.name,
    }
    for part_manufacturer in many_to_many_temp
    for part in parts
    if part.id == part_manufacturer["part_id"]
]

third_task(many_to_many)

if __name__ == "__main__":
    main()

```

```

"""Domain entities module for task implementation algorithm.

This is module provides domain subject enteties.

Classes:
    Part: Represents a manufacturing part with attributes.
    Manufacturer: Represents a manufacturer with attributes.
"""

from .part import Part
from .manufacturer import Manufacturer

__all__ = ["Part", "Manufacturer"]
__version__ = "0.1.0"

def get_version():
    return __version__

```

```

"""Module for representing manufacturers."""

class Manufacturer:
    """Represents a manufacturer.

    Attributes:
        id: Unique identifier for the manufacturer.
        name: Name of the manufacturer.
        industry: Industry sector of the manufacturer.
        specialization: Specialization area of the manufacturer.
        production_capacity: Production capacity in units per time period.
    """

```

```

        phone: Contact phone number.
        email: Contact email address.
    """

    def __init__(
        self,
        manufacturer_id: int,
        name: str,
        industry: str,
        specialization: str,
        production_capacity: int,
        phone: str,
        email: str,
    ):
        self.id = manufacturer_id
        self.name = name
        self.industry = industry
        self.specialization = specialization
        self.production_capacity = production_capacity
        self.phone = phone
        self.email = email

    def __eq__(self, other):
        """Checks equality based on manufacturer attributes."""
        if not isinstance(other, type(self)):
            return NotImplemented
        return (
            self.name == other.name
            and self.industry == other.industry
            and self.specialization == other.specialization
            and self.production_capacity == other.production_capacity
            and self.email == other.email
            and self.phone == other.phone
        )

    def __hash__(self):
        """Returns hash based on manufacturer attributes."""
        return hash(
            (
                self.name,
                self.industry,
                self.specialization,
                self.production_capacity,
                self.phone,
                self.email,
            )
        )

    def __lt__(self, other):
        """Compares manufacturers based on production capacity."""
        if not isinstance(other, type(self)):

```

```
        return NotImplemented
    return self.production_capacity < other.production_capacity
```

```
"""Module for representing manufacturing parts."""

class Part:
    """Represents a manufacturing part.

    Attributes:
        id: Unique identifier for the part.
        manufacturer_id: identifier for the manufacturer.
        name: Name of the part.
        weight: Weight of the part in grams.
        category: Category of the part.
        production_cost: Production cost in currency units.
    """

    def __init__(
        self,
        part_id: int,
        manufacturer_id: int,
        name: str,
        weight: int,
        category: str,
        production_cost: int,
    ):
        self.id = part_id
        self.manufacturer_id = manufacturer_id
        self.name = name
        self.weight = weight
        self.category = category
        self.production_cost = production_cost

    def __eq__(self, other):
        """Checks equality based on part attributes."""
        if not isinstance(other, type(self)):
            return NotImplemented
        return (
            self.manufacturer_id == other.manufacturer_id
            and self.name == other.name
            and self.weight == other.weight
            and self.category == other.category
            and self.production_cost == other.production_cost
        )

    def __hash__(self):
        """Returns hash based on part attributes."""
        return hash(
```

```
        (
            self.manufacturer_id,
            self.name,
            self.weight,
            self.category,
            self.production_cost,
        )
    )

def __lt__(self, other):
    """Compares parts based on production cost."""
    if not isinstance(other, type(self)):
        return NotImplemented
    return self.production_cost < other.production_cost
```